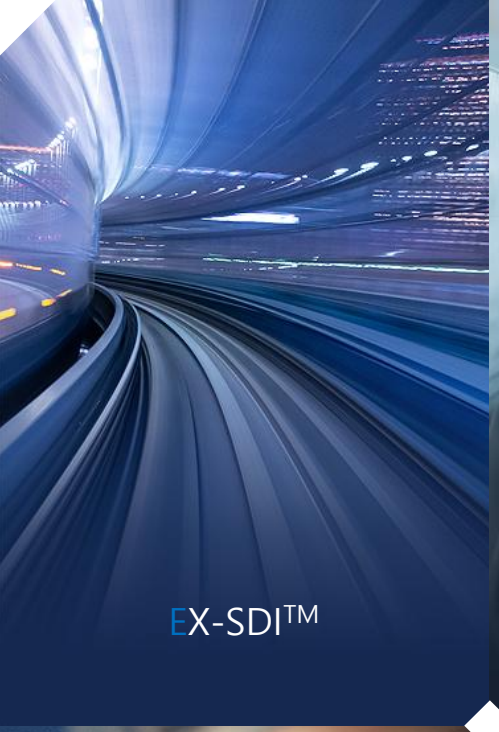
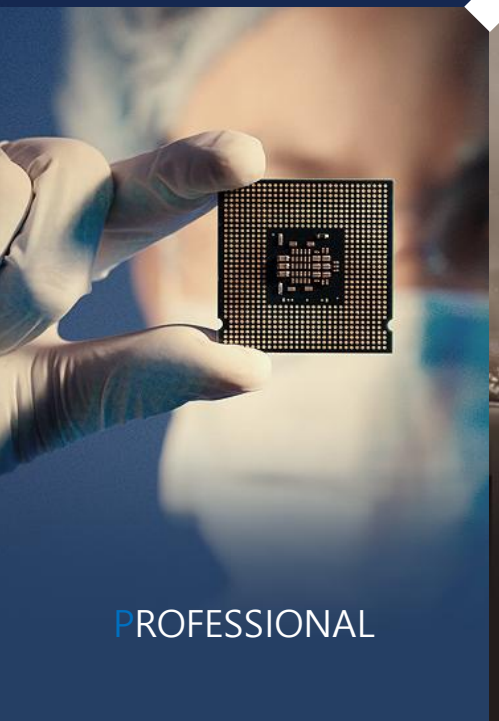
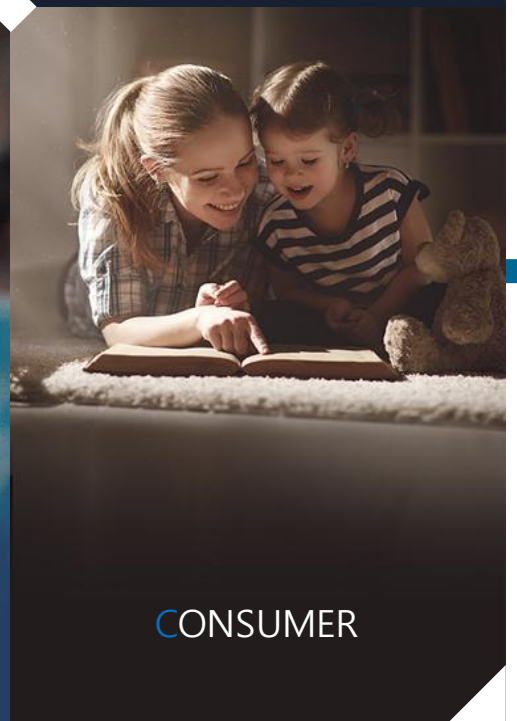






eyenix
the great eyes

OJT ASIC Part

2025.12.31

서윤철

CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

목차

- LAB0
  - 설계
  - 검증
  - 추가 응용
- LAB4
  - 설계
  - 검증
  - 추가 응용
- 마무리

CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

실습환경

Tool	Vivado
HDL	Verilog, SystemVerilog

CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

LAB0

SPEC

- Asynchronous FIFO(First In First Out) 설계
  - 입력 설정은 파라미터 형태
  - Register memory 사용
  - 추가 기능 자유롭게
  - 검증 방법론 각자 알아서 진행
- + 추가 응용
  - 1:N, N:1 형태의 Asynchronous FIFO 설계
  - Ex) Ratio: N=1,2,3....16

- Parameters

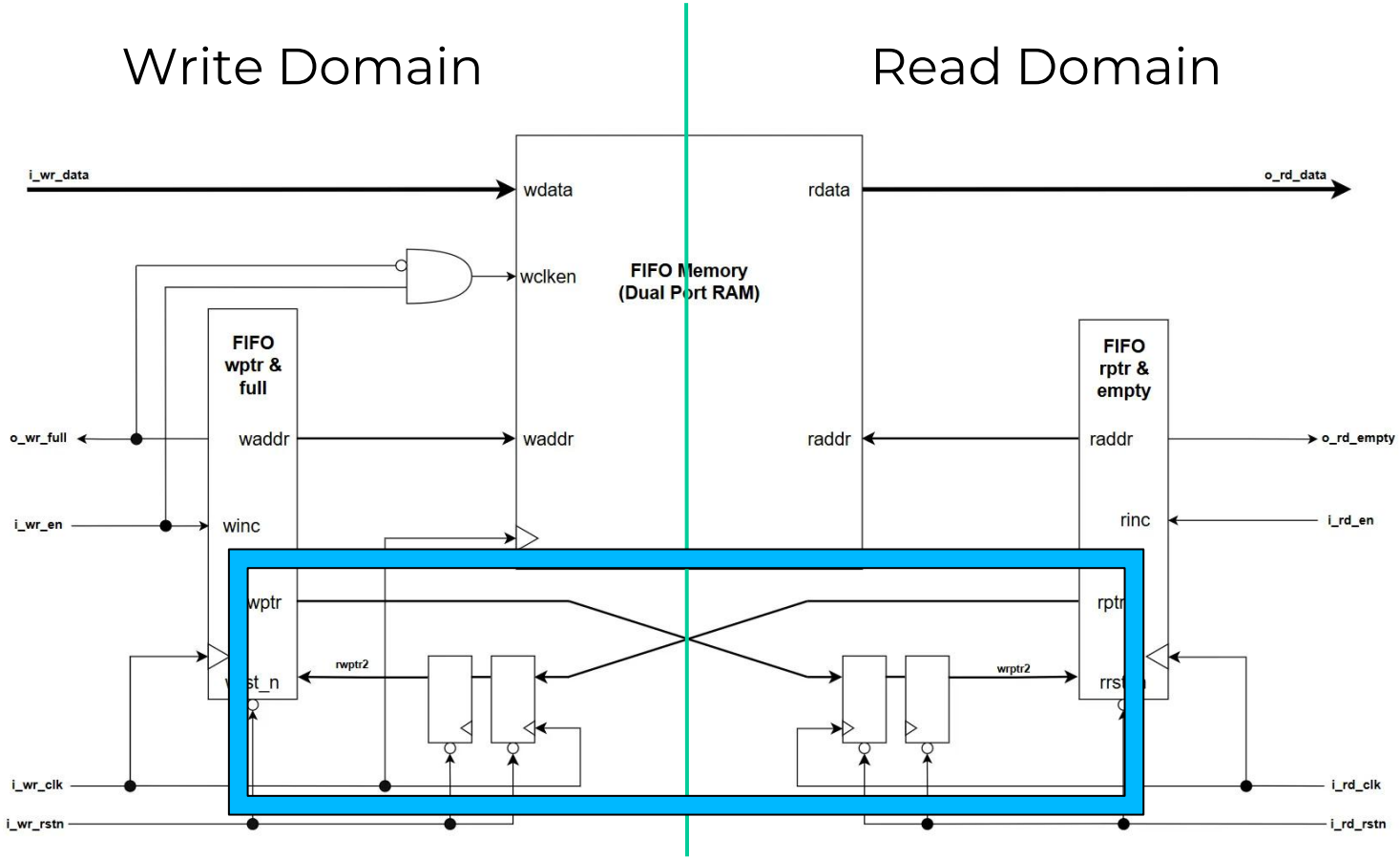
WIDTH	Register Memory width
DEPTH	Register Memory Depth
PFULL_TH	Program Full Threshold
PEMPTY_TH	Program Empty Threshold
DEBUG_MODE	DEBUG MODE

CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

Asynchronous FIFO 설계(1:1)

- 안정성 향상
  - CDC(Clock Domain Crossing)
    - 2단 동기화(2-FF sync)
  - Glitch
    - Gray Code

Meta stability 방지



- Block Diagram

CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

Asynchronous FIFO 검증(1:1)

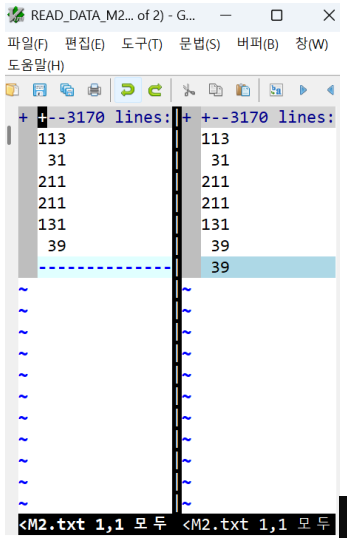
- 디버그 모듈(log 출력)
- 기능 검증(N: 100)
  - 데이터 무결성
- 신뢰성 검증(N: 10k, 100k, 1M)
  - 데이터 무결성
  - 리셋 테스트

"Write Data(.txt) = Read Data(.txt)"

N: fork join을 반복하는 횟수

Clock Frequency(MHz)(write, read)		
(125, 400)	(125, 200)	(125, 125)
(200, 400)	(200, 200)	(200, 125)
(400, 400)	(400, 200)	(400, 125)

방식	내용
데이터 무결성	1. Write를 FIFO가 full이 될 때까지 반복한다. 2. Read를 10번 실행하여 FIFO가 full이 아닌 상태로 만든다. 3. Fork join_any을 실행시켜 N회 write와 read를 반복한다. 4. Fork join이 끝나면 FIFO가 empty상태가 될 때까지 read한다.
리셋 테스트	데이터 무결성 검증 방식과 유사하지만, 어떠한 시점 혹은 여러 차례 리셋을 실행시켜 설계한 FIFO가 잘 동작하는지 확인한다.




```
gvim -d .\READ_DATA.txt .\WRITE_DATA.txt
```


CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

eyenix
the great eyes

Asynchronous FIFO 검증(1:1)

Clock Frequency(MHz)(write, read)		
(125, 400)	(125, 200)	(125, 125)
(200, 400)	(200, 200)	(200, 125)
(400, 400)	(400, 200)	(400, 125)

- 신뢰성 검증(WIDTH: 32, DEPTH: 5, N: 10k, 100k, 1M)
  - 데이터 무결성
    - (Write, Read): (400, 125)

N	10k	100k	1M
Write trial	10032	100032	1000032
Write success	3166	31291	312541
Write fail	6866	68741	687491
Read trial	3166	31291	312541
Read success	3166	31291	312541
Read fail	0	0	0
Diff between Write & Read	Same	Same	Same

    - (Write, Read): (125, 400)

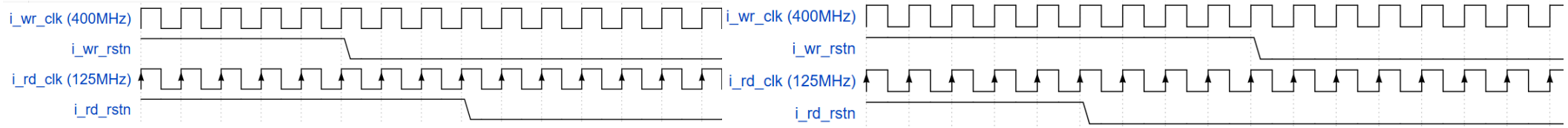
N	10k	100k	1M
Write trial	3158	31283	312533
Write success	3158	31283	312533
Write fail	0	0	0
Read trial	10012	100012	1000012
Read success	3157	31282	312532
Read fail	6855	68730	687480
Diff between Write & Read	Same	Same	Same

CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

Clock Frequency(MHz)(write, read)		
(125, 400)	(125, 200)	(125, 125)
(200, 400)	(200, 200)	(200, 125)
(400, 400)	(400, 200)	(400, 125)

Asynchronous FIFO 검증(1:1)

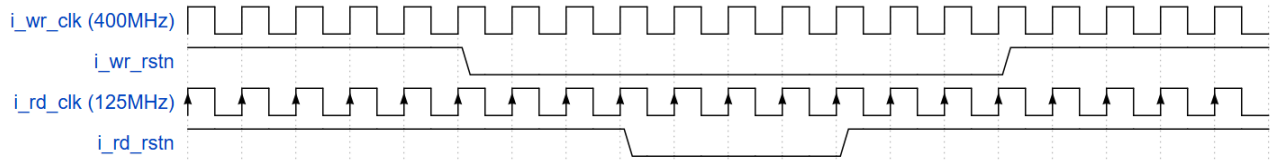
- 신뢰성 검증(WIDTH: 32, DEPTH: 5, N: 10k)
  - 리셋 테스트
    - Test Case 1
      1. write domain 먼저 리셋 해제, 이후 read domain 리셋 해제
      2. read domain 먼저 리셋 해제, 이후 write domain 리셋 해제



    - Test Case 2

      : 정상 동작 중에 리셋(단, write와 read 사이에 리셋 상태가 겹치는 구간 존재)

      1. 리셋 시점은 서로 다르게
      2. 리셋 길이도 서로 다르게



CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

Asynchronous FIFO 검증(1:1)

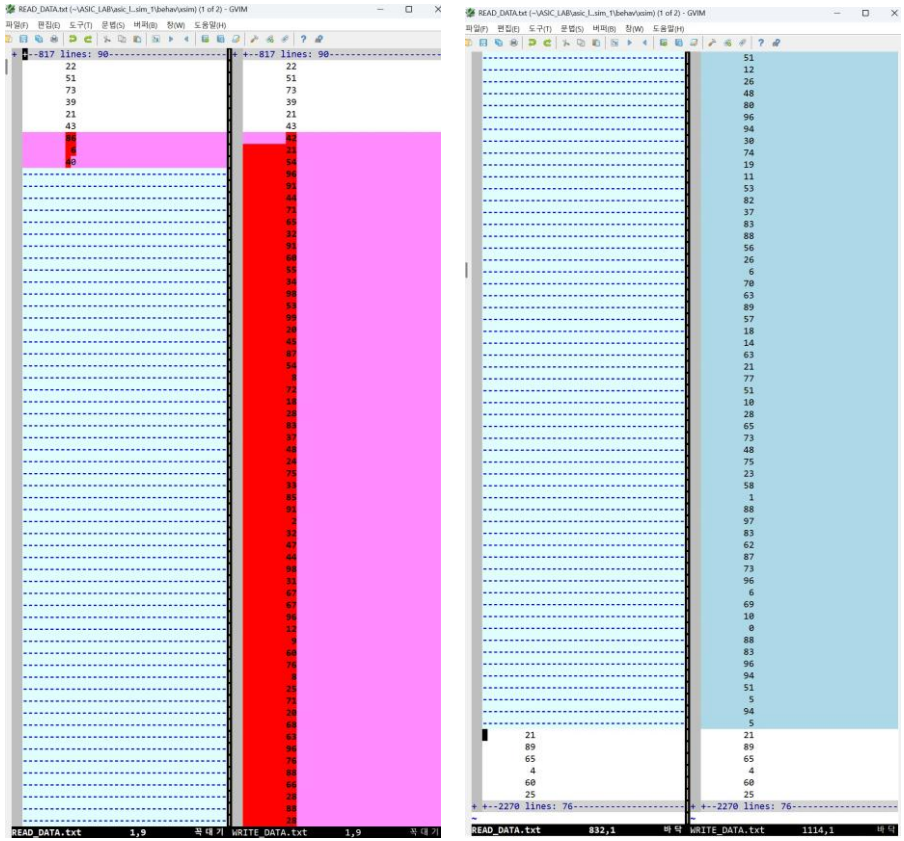
Clock Frequency(MHz)(write, read)		
(125, 400)	(125, 200)	(125, 125)
(200, 400)	(200, 200)	(200, 125)
(400, 400)	(400, 200)	(400, 125)

- 신뢰성 검증(WIDTH: 32, DEPTH: 5, N: 10k)
  - 리셋 테스트
    - Test Case 1

Case	1	2
Write trial	10032	9988
Write success	3151	3151
Write fail	6881	6837
Read trial	3153	3166
Read success	3151	3151
Read fail	2	15

    - Test Case 2

Write trial	7187
Write success	2276
Write fail	4911
Read trial	2315
Read success	2276
Read fail	39



CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

Asynchronous FIFO(1:1) 합성 결과

- Parameters
  - WIDTH: 64
  - DEPTH: 11 (= 2^11)

- Slice Logic

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	1730	0	0	41000	4.22
LUT as Logic	354	0	0	41000	0.86
LUT as Memory	1376	0	0	13400	10.27
LUT as Distributed RAM	1376	0			
LUT as Shift Register	0	0			
Slice Registers	151	0	0	82000	0.18
Register as Flip Flop	151	0	0	82000	0.18
Register as Latch	0	0	0	82000	0.00
F7 Muxes	128	0	0	20500	0.62
F8 Muxes	64	0	0	10250	0.62
Unique Control Sets	18		0	10250	0.18

- IO and GT Specific

Site Type	Used	Fixed	Prohibited	Available	Util%
Bonded IOB	354	0	0	300	118.00
Bonded IPADs	0	0	0	26	0.00
Bonded OPADs	0	0	0	16	0.00
PHY_CONTROL	0	0	0	6	0.00
PHASER_REF	0	0	0	6	0.00
OUT_FIFO	0	0	0	24	0.00
IN_FIFO	0	0	0	24	0.00
IDELAYCTRL	0	0	0	6	0.00
IBUFDS	0	0	0	288	0.00
GTXE2_COMMON	0	0	0	2	0.00
GTXE2_CHANNEL	0	0	0	8	0.00
PHASER_OUT/PHASER_OUT_PHY	0	0	0	24	0.00
PHASER_IN/PHASER_IN_PHY	0	0	0	24	0.00
IDELAYE2/IDELAYE2_FINEDELAY	0	0	0	300	0.00
ODELAYE2/ODELAYE2_FINEDELAY	0	0	0	100	0.00
IBUFDS_GTE2	0	0	0	4	0.00
ILOGIC	0	0	0	300	0.00
OLOGIC	0	0	0	300	0.00

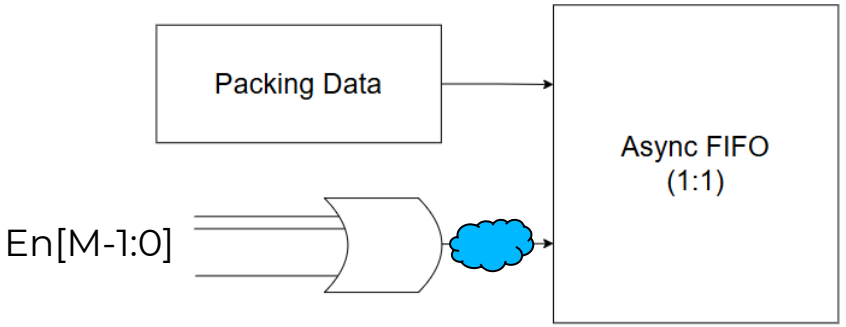
- Primitives

Ref Name	Used	Functional Category
RAMD64E	1376	Distributed Memory
LUT6	291	LUT
OBUFT	192	IO
FDCE	149	Flop & Latch
MUXF7	128	MuxFx
OBUF	92	IO
IBUF	70	IO
MUXF8	64	MuxFx
LUT2	31	LUT
LUT3	25	LUT
CARRY4	20	CarryLogic
LUT5	13	LUT
LUT4	13	LUT
LUT1	2	LUT
FDPE	2	Flop & Latch
BUFG	2	Clock

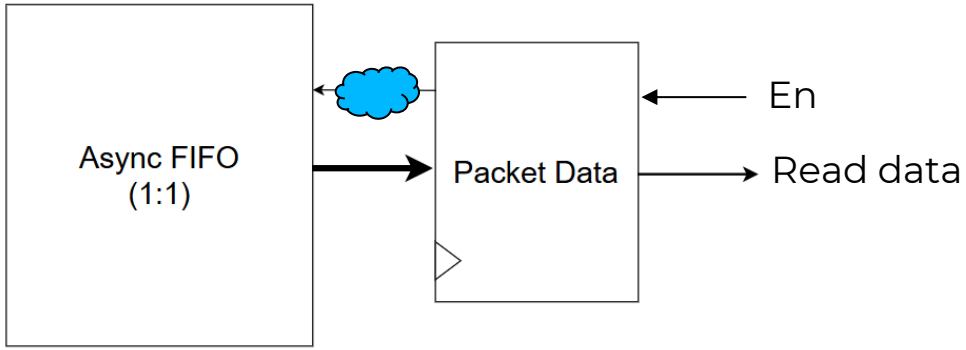
CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

LAB0 + 추가 응용

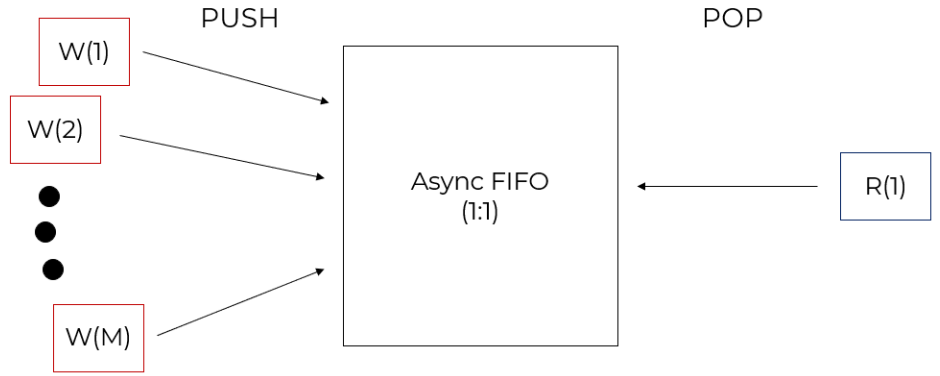
- M:1 async FIFO 설계
  - Write Control



  - Read Control



(조건1) Data width of a writer == Data width of a reader
(조건2) Writers 각각의 Data width 동일



W(M) ..... W(2) W(1)
Packing Data

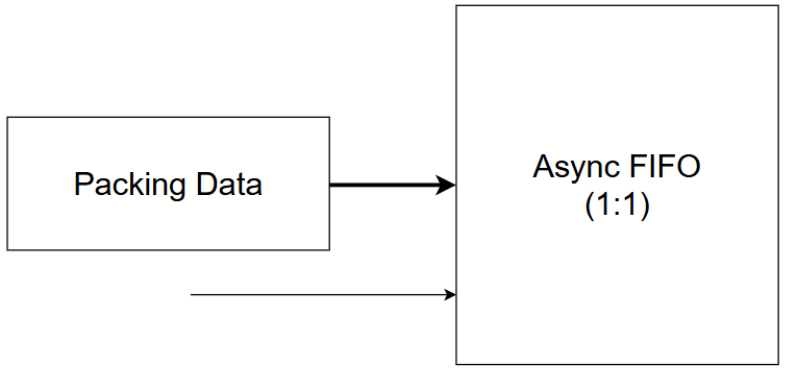
기존 Async FIFO(1:1)와 차이점: wr_en가 다수이다.
모든 Writers가 write하지 않는 경우도 고려해야 함.
Async FIFO의 Register Memory의 비트에 유효한 데이터인지 확인할 수 있는 마스크 비트 추가 선언

M-bit	W(M) ..... W(2) W(1)
MASK	DATA

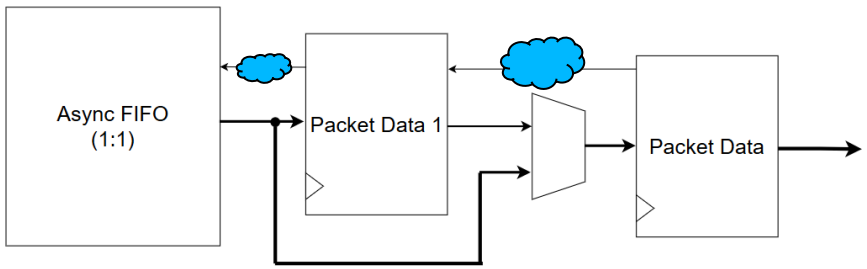
CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

LAB0 + 추가 응용

- 1:N async FIFO 설계
  - Write Control



  - Read Control



Data width of a writer == Data width sum of readers
Readers 각각의 Data width 동일
Packing Data는 모두 유효한 값

W(N) ..... W(2) W(1)
Packing Data

Packet register가 두개 존재하는 이유:

- 만약 Packet register가 하나만 존재한다면?
현재 클럭에 readers가 packet data를 pop했지만 packet data가 empty하지 않는 경우 && 다음 클럭에 packet register에 남아있는 데이터보다 readers가 더 많이 pop을 요청
(문제) 각각의 readers에게 공정하게 data를 분배하지 못함.
(해결) packet register를 하나 더 배치하여 위의 문제를 사전 차단!

CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

LAB0 + 추가 응용

- M:1 async FIFO 검증

M_Writers	2	3	4	5	6	7	8	9	14	15	16
Write trial	7270	8819	9438	9767	9892	9946	9988	10012	0033	10033	10034
Write success	3177	3187	3204	3220	3243	3252	3267	3289	380	3387	3420
Read trial	3176	3185	3201	3218	3240	3246	3260	3283	366	3374	3404
Read success	3176	3185	3201	3218	3240	3246	3260	3283	366	3374	3404

- 1:N async FIFO 검증

N_Readers	2	3	4	5	6	7	8	9	14	15	16
Write trial	34 + 10k (WRITE DATA WIDTH:										
Write success	1216	1268	1314	1356	1389	1424	1438	1450	496	1508	1530
Read trial	3302	4919	6548	8123	9763	11344	12921	14553	2528	24165	25759
Read success	2431	3800	5251	6775	8324	9960	11493	13043	0933	22602	24462

```
# run_sweep_xsim.tcl
set here [file dirname [info script]]
cd $here

set Ms {2 3 4 5 6 7 8 9 10 11 12 13 14 15 16}
#set Ns {2 3 4 5 6 7 8 9 10 11 12 13 14 15 16}

# include paths
set INC_TB  "../asic_lab-main/LAB_0/SIM/TB"
set INC_LIB "../asic_lab-main/LAB_0/LIB"

# source files
set TB    "../asic_lab-main/LAB_0/SIM/TB/tb_main.sv"
set DUT   "./asic_lab0.srcs/sources_1/new/MtoN_async_fifo.sv"
set ASYNC "../asic_lab-main/LAB_0/RTL/async_fifo.v"

foreach M $Ms {
    puts "============================="
    puts " RUN : M_WRITERS = $M"
    puts "============================="

    # compile
    exec xvlog -sv \
        -i $INC_TB \
        -i $INC_LIB \
        $TB $DUT $ASYNC

    # elaborate (⭐ 핵심: -generic)
    exec xelab -debug typical \
        -top tb_main \
        -snapshot sim_M${M} \
        -generic M_WRITERS=$M

    # run
    exec xsim sim_M${M} -runall -log log_M${M}.txt
}

#foreach N $Ns {
#    puts "============================="
#    puts " RUN : N_READERS = $N"
#    puts "============================="
#
#    # compile
#    exec xvlog -sv \
#        -i $INC_TB \
#        -i $INC_LIB \
#        $TB $DUT $ASYNC
#
#    # elaborate (⭐ 핵심: -generic)
#    exec xelab -debug typical \
#        -top tb_main \
#        -snapshot sim_N${N} \
#        -generic N_READERS=$N
#
#    # run
#    exec xsim sim_N${N} -runall -log log_N${N}.txt
#}
puts "==== ALL SWEEP DONE ===="
```


COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

LAB4

SPEC

- Video timing parameters를 적용하여 Sync Generator 설계
  - 이미지 Test Pattern 형태
  - 검증: /LIB/img_save.sv 모듈 frame 저장

- + 추가 응용
  - (SYNC_GEN ===➔ (OSD_BOX) ===➔ IMG_SAVE)
    - osd_box 모듈을 이용해서 원하는 위치에 박스 그리기
    - 박스 색
    - 박스 형태
    - 알파블랜딩 (투명도)

CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

Ports

Input	i_clk, i_rstn, i_init_done
Output	o_hsync, o_vsync, o_pixel_data, o_pixel_valid, o_data_type(6'h1E)

Parameters

- Video timing parameters

  HACT, VACT, HSA, HBP, HFP, VSA, VBP, VFP

- Plus

  IMG_MODE, BOX_MODE, BOX_LO_H, BOX_LO_V, BOX_SIZE_H, BOX_SIZE_V, BOX_COLOR, BOX_LINE, APLHA_BLENDING

Pixel Format

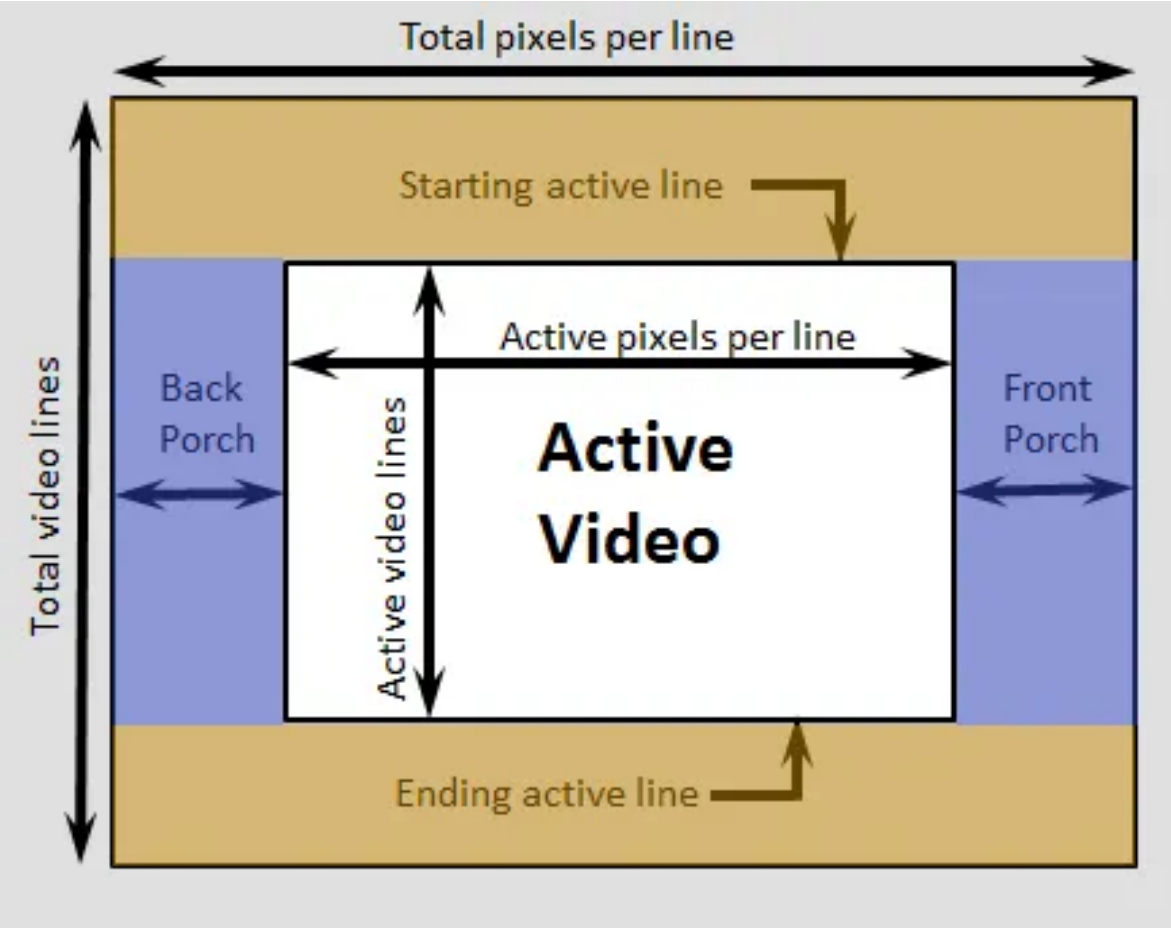
- YUV422 8 bit

63 56	55 48	47 40	39 32	31 24	23 16	15 8	7 0
Pixel 4		Pixel 3		Pixel 2		Pixel 1	
Y4	V3	Y3	U3	Y2	V1	Y1	U1

4 Pixels for Clock

CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

Video timing parameters 를 적용하여 Sync Generator 설계



Video Timing Parameters

The following waveforms show the video interface signals relationship.

Figure 2: Video Timing Waveform (Horizontal)

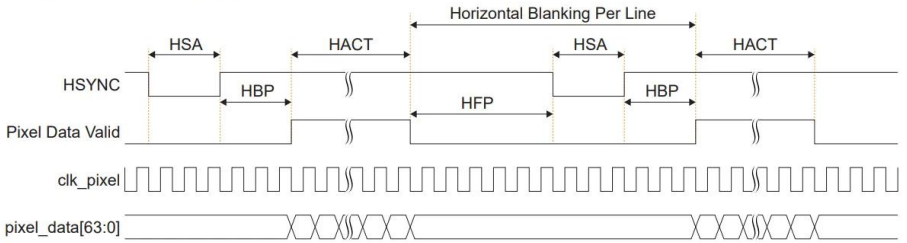


Figure 3: Video Timing Waveform (Vertical)

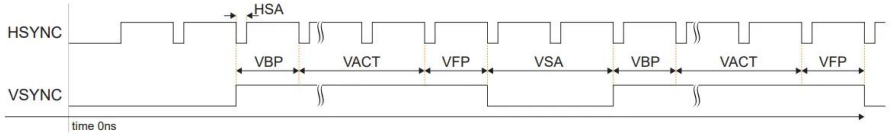


Table 34: Video Timing Parameter Definitions

MIPI Video Timing Parameters	Definition	Min	Max	Unit
HACT	Total number of pixel per line	16	8,192	Pixel
VACT	Total number of line per frame	1	8,192	Line
HSA	HSYNC pulse width	1	4,096	Pixel
HBP	Horizontal back porch	1	4,096	Pixel
HFP	Horizontal front porch	1	4,096	Pixel
VSA	VSYNC pulse width	1	8,192	Line
VBP	Vertical back porch	1	8,192	Line
VFP	Vertical front porch	1	8,192	Line
Pixel Clock	Video stream pixel clock frequency in MHz	(6)	(6)	MHz
MIPI Speed	CSI-2 TX MIPI speed in Mbps	80	2,500	Mbps
No. data lane	Number of MIPI data lane	1	4	Lane

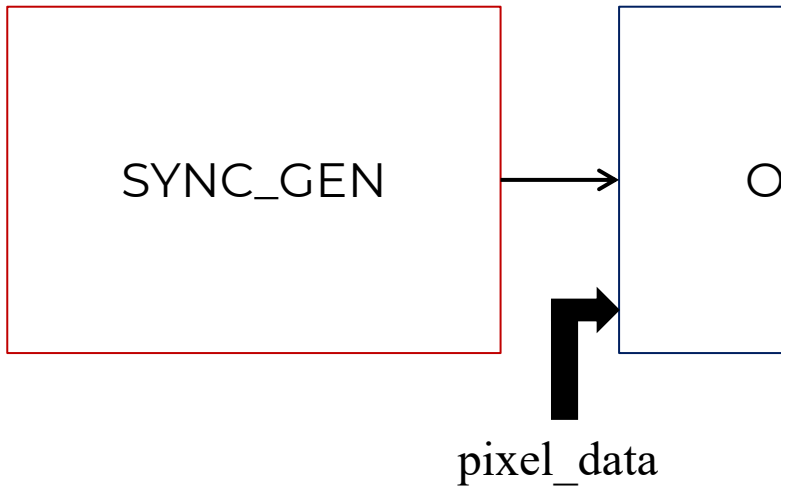
ALL RIGHTS RESERVED

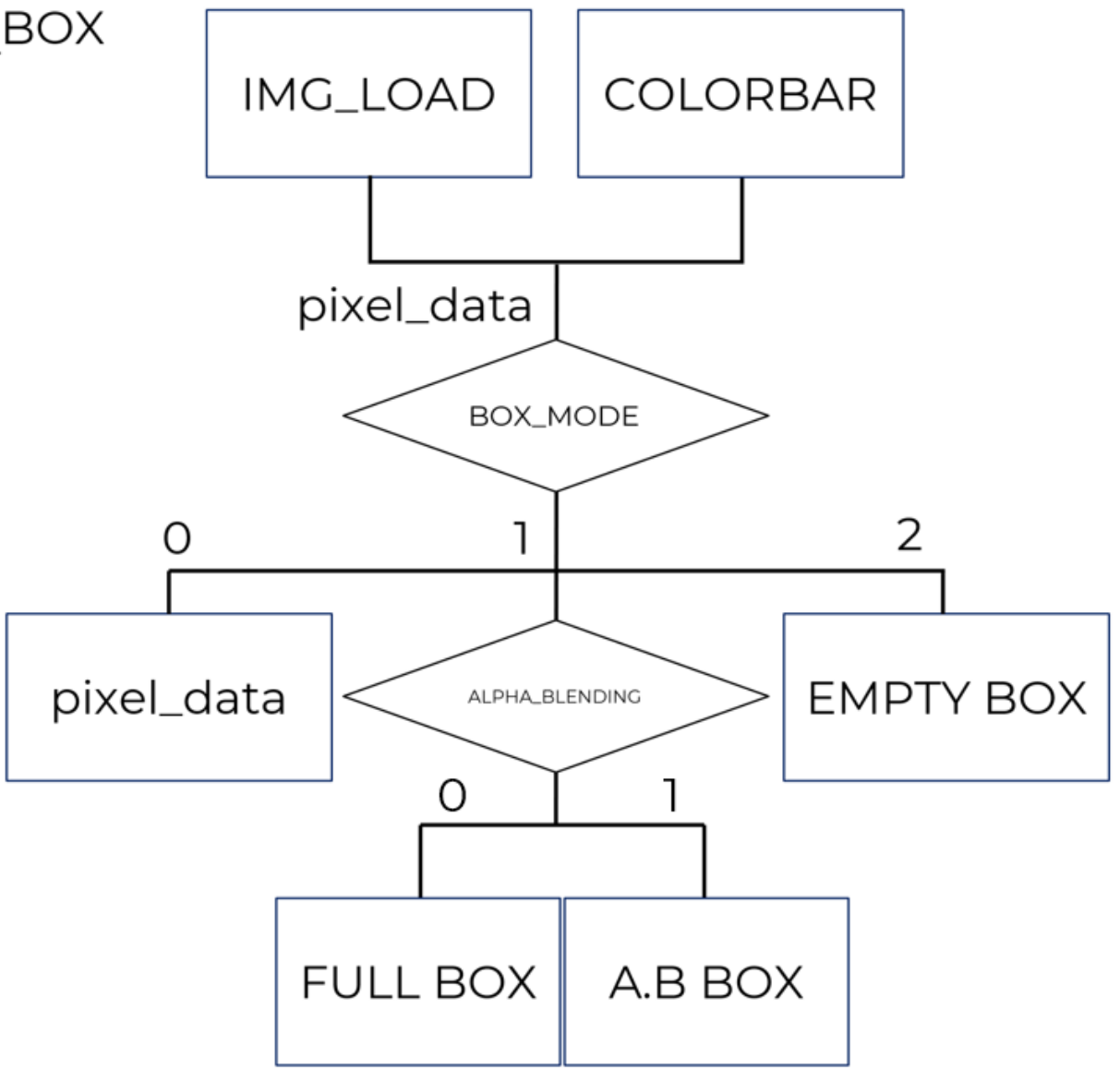
LAB4

실습 진행

1. COLORBAR 모듈을 이용해 sync gener
2. IMG_LOAD 모듈을 이용해 복잡한 이미지
3. OSD_BOX에 대한 실습

블록 다이어그램





실습 결과

1. COLORBAR 모듈을 이용해 sync generator가 정확히 동작하는지 확인

2. IMG_LOAD 모듈을 이용해 복잡한 이미지를 잘 처리할 수 있는지 확인

Google Colab Script(.py)

18 PNG 이미지(RGB)를 (.yuv) 형식으로 변환한 뒤, 64bit YUV422 형식에 맞춰 hex값으로 변환 -> IMG_LOAD 모듈 내에서 readmemh

CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

LAB4 + 추가 응용

실습 결과

3. OSD_BOX에 대한 실습

```
`define WHITE (64'hEB80EB80EB80EB80)
`define YELLOW (64'hD292D210D292D210)
`define SKYBLUE (64'hAA10AAA6AA10AAA6)
`define GREEN (64'h9122913691229136)
`define PINK (64'h6ADE6ACA6ADE6ACA)
`define RED (64'h51F0515A51F0515A)
`define BLUE (64'h296E29F0296E29F0)
`define BLACK (64'h1080108010801080)
```

속이 채워진 박스


```
parameter HACT = 1920;
parameter VACT = 1080;
parameter HSA = 4;
parameter HBP = 4;
parameter HFP = 4;
parameter VSA = 1;
parameter VBP = 1;
parameter VFP = 1;

parameter IMG_MODE = 1;
parameter BOX_MODE = 1;
parameter BOX_LO_H = 500;
parameter BOX_LO_V = 840;
parameter BOX_SIZE_H = 1000;
parameter BOX_SIZE_V = 150;
parameter BOX_COLOR = `GREEN;
parameter BOX_LINE = 20;
parameter ALPHA_BLENDING = 1;
```

속이 빈 박스

투명 박스

박스 위치 조정

CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED

마무리

진행 시 막혔던 부분 & 아쉬운 점

- 구체적인 스펙이 없는 추가 응용(M:N Async FIFO)

➔ 구현 방법, 응용 분야가 다양하여 선택하는 것에 대한 어려움

➔ 현재 방식 선택 이유: ASIC_LAB의 README.md에서 다양한 BUS protocol 및 LAB4의 Video Interface에 필요한 FIFO가 무엇일까 생각하다가 현재 방식 채택!

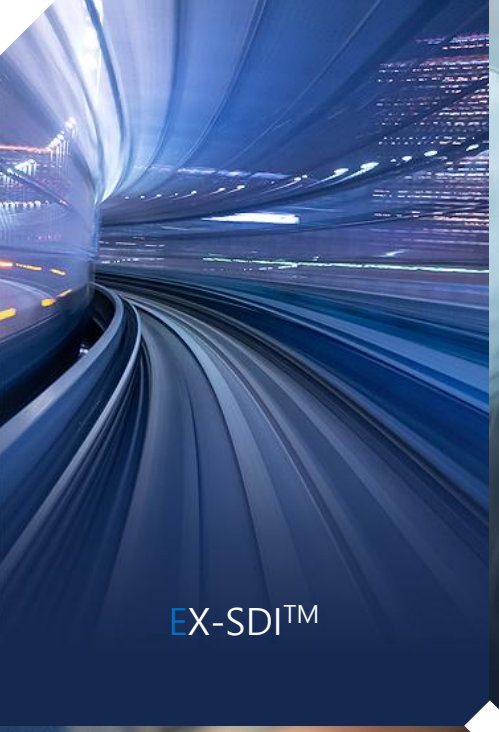
- 다양한 테스트 케이스에 대한 고민

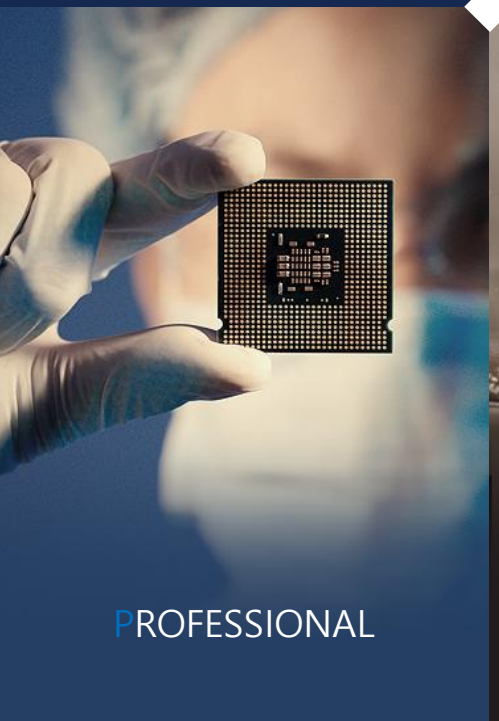
도움을 받은 사이트: ChatGPT

Clifford E. Cummings, *Simulation and Synthesis Techniques for Asynchronous FIFO Design*, Sunburst Design, Inc.

MIPI Alliance, MIPI CSI-2 Specification, Video Timing Parameters.

CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED





eyenix
the great eyes

Thank You!

The great eyes of the world!

CONFIDENTIAL – COPYRIGHT 2021 EYENIX ALL RIGHTS RESERVED